of programs it will take a huge amount of time, because the number of combinations increases exponentially with the number of threads and the number of operations performed by each thread. This technique is thus best reserved for fine-grained tests of individual pieces of code rather than an entire application. The other obvious downside is that it relies on the availability of simulation software that can handle the operations used in your code.

So, you have a technique that involves running your test many times under normal conditions but that might miss problems, and you have a technique that involves running your test many times under special conditions but that's more likely to find any problems that exist. Are there any other options?

A third option is to use a library that detects problems as they occur in the running of the tests.

Detecting Problems Exposed by Tests with a Special Library

Although this option doesn't provide the exhaustive checking of a combination simulation test, you can identify many problems by using a special implementation of the library synchronization primitives such as mutexes, locks, and condition variables. For example, it's common to require that all accesses to a piece of shared data be done with a particular mutex locked. If you could check which mutexes were locked when the data was accessed, you could verify that the appropriate mutex was indeed locked by the calling thread when the data was accessed and report a failure if this was not the case. By marking your shared data in some way, you can allow the library to check this for you.

Such a library implementation can also record the sequence of locks if more than one mutex is held by a particular thread at once. If another thread locks the same mutexes in a different order, this could be recorded as a *potential* deadlock even if the test didn't actually deadlock while running.

Another type of special library that could be used when testing multithreaded code is one where the implementations of the threading